

# CSS Combinators & Pseudo-classes

*Detailed Notes — Selectors, Syntax, Examples & Specificity*

## Part 1 — CSS Combinators

A **combinator** is a symbol placed between two CSS selectors that explains the relationship between them — i.e. which elements are matched based on how they are positioned relative to each other in the HTML tree. CSS defines four combinators.

| Symbol | Name                        | Matches                                                                         |
|--------|-----------------------------|---------------------------------------------------------------------------------|
| A B    | Descendant combinator       | Any B that is nested anywhere inside A (child, grandchild, etc.)                |
| A > B  | Child combinator            | Any B that is a direct (immediate) child of A                                   |
| A + B  | Adjacent sibling combinator | The B that comes immediately after A, sharing the same parent                   |
| A ~ B  | General sibling combinator  | Any B that comes after A (not necessarily immediately), sharing the same parent |

### 1.1 Descendant Combinator ( space )

Syntax: A B — selects every <B> element that is a descendant of <A>, no matter how deeply nested. This is the most commonly used combinator.

```
article p {  
  color: #333;  
}  
  
/* Matches ALL <p> tags inside <article>,  
even if wrapped in other elements */
```

**HTML example matched:**

```
<article>  
<p>Matched</p>  
<div><p>Also matched (nested deeper)</p></div>  
</article>
```

### 1.2 Child Combinator ( > )

Syntax: A > B — selects only the <B> elements that are **direct children** of <A>. Grandchildren are NOT matched.

```
ul > li {  
  list-style: square;  
}  
  
/* Matches direct <li> children of <ul>,  
but NOT an <li> nested inside a sub <ul> */
```

## 1.3 Adjacent Sibling Combinator ( + )

Syntax: A + B — selects the <B> element that is placed **immediately after** <A>, and both must share the same parent. Only ONE element is matched.

```
h2 + p {  
margin-top: 0;  
font-weight: bold;  
}  
  
/* Only the very first <p> right after an <h2>  
gets this style */
```

## 1.4 General Sibling Combinator ( ~ )

Syntax: A ~ B — selects **all** <B> siblings that come after <A> (not necessarily immediately after), as long as they share the same parent.

```
h2 ~ p {  
color: grey;  
}  
  
/* Every <p> that appears after the <h2>  
at the same level gets this style */
```

## 1.5 Column Combinator ( || ) — newer / limited support

Syntax: A || B — part of the CSS Table/Grid spec, used to select cells that belong to a column defined via <col>. Browser support is very limited, so it is rarely used in production.

```
col.price || td {  
background: #eef;  
}
```

### Quick Comparison

| Selector | Meaning in plain words                |
|----------|---------------------------------------|
| div p    | any p anywhere inside div             |
| div > p  | p that is a direct child of div       |
| div + p  | p immediately after div (same parent) |
| div ~ p  | every p after div (same parent)       |

*Tip: Combinators can be chained together to build very specific selectors, e.g. section > article h2 + p reads as: a <p> right after an <h2>, where that <h2> is somewhere inside an <article> that is a direct child of a <section>.*

## Part 2 — CSS Pseudo-classes

A **pseudo-class** is a keyword added to a selector, preceded by a single colon (:), that selects elements based on a **state**, **position**, or **relationship** that isn't expressible with a plain selector alone (e.g. being hovered, being the first child, being checked).

```
selector:pseudo-class {  
  property: value;  
}
```

```
/* Example */  
a:hover {  
  color: red;  
}
```

*Pseudo-class (single colon :) vs Pseudo-element (double colon ::): a pseudo-class targets an existing element in a certain state (e.g. :hover), while a pseudo-element targets a sub-part of an element that doesn't exist as a real DOM node (e.g. ::before, ::first-line). This note focuses only on pseudo-classes.*

### 2.1 Link & User-Action Pseudo-classes

| Pseudo-class   | Description                                                       | Common Use                                       |
|----------------|-------------------------------------------------------------------|--------------------------------------------------|
| :link          | Unvisited links                                                   | a:link { color: blue; }                          |
| :visited       | Already visited links                                             | a:visited { color: purple; }                     |
| :hover         | Element under the mouse pointer                                   | button:hover { background: #eee; }               |
| :active        | Element being clicked/activated                                   | button:active { transform: scale(0.97); }        |
| :focus         | Element that currently has keyboard focus                         | input:focus { outline: 2px solid blue; }         |
| :focus-visible | Focused AND focus should be visibly indicated (e.g. via keyboard) | btn:focus-visible { outline: 2px solid orange; } |
| :focus-within  | Element that contains a focused descendant                        | form:focus-within { border-color: blue; }        |

**Order matters!** The classic LVHA order for links must be respected or styles may not apply correctly: :link → :visited → :hover → :active.

```
:link { color: blue; }  
:visited { color: purple; }  
:hover { color: orange; }  
:active { color: red; }
```

### 2.2 Structural / Positional Pseudo-classes

| Pseudo-class | Description                                   |
|--------------|-----------------------------------------------|
| :first-child | Element that is the first child of its parent |
| :last-child  | Element that is the last child of its parent  |

| Pseudo-class         | Description                                                                      |
|----------------------|----------------------------------------------------------------------------------|
| :only-child          | Element that is the ONLY child of its parent                                     |
| :first-of-type       | First sibling of its element type                                                |
| :last-of-type        | Last sibling of its element type                                                 |
| :only-of-type        | Only sibling of its element type                                                 |
| :nth-child(n)        | Matches the nth child — n can be a number, keyword (odd/even), or formula (an+b) |
| :nth-last-child(n)   | Same as nth-child but counts from the end                                        |
| :nth-of-type(n)      | Matches the nth sibling of its type                                              |
| :nth-last-of-type(n) | Same as nth-of-type but counts from the end                                      |
| :root                | The document's root element (i.e. <html>)                                        |
| :empty               | Element with no children (including no text nodes)                               |

```
li:first-child { font-weight: bold; }
li:nth-child(2) { color: teal; }
li:nth-child(odd){ background: #f7f7f7; }
li:nth-child(3n) { border-top: 1px solid #ccc; } /* every 3rd item */
li:nth-child(n+3){ opacity: 0.8; } /* 3rd item onward */
```

**nth-child formula:**  $an + b$  — for each non-negative integer  $n$  (0,1,2,3...), the element at position  $(an + b)$  is selected. Example:  $3n = 3,6,9...$   $3n+1 = 1,4,7...$

## 2.3 Form & Input-State Pseudo-classes

| Pseudo-class   | Description                                                              |
|----------------|--------------------------------------------------------------------------|
| :checked       | A checked checkbox, radio button, or selected <option>                   |
| :indeterminate | Checkbox/radio in an indeterminate (neither checked nor unchecked) state |
| :disabled      | Form element that is disabled                                            |
| :enabled       | Form element that is enabled (default, interactive)                      |
| :required      | Input with the 'required' attribute                                      |
| :optional      | Input WITHOUT the 'required' attribute                                   |
| :valid         | Input whose value passes validation                                      |
| :invalid       | Input whose value fails validation                                       |
| :in-range      | Numeric input whose value is within min/max range                        |
| :out-of-range  | Numeric input whose value is outside min/max range                       |

| Pseudo-class       | Description                                                    |
|--------------------|----------------------------------------------------------------|
| :read-only         | Element that cannot be edited by the user                      |
| :read-write        | Element that IS editable (e.g. text input, contenteditable)    |
| :placeholder-shown | Input currently showing its placeholder text (empty)           |
| :default           | The default option/button in a group (e.g. pre-selected radio) |

```
input:required { border-left: 3px solid crimson; }
input:invalid { border-color: red; }
input:checked + label { font-weight: bold; color: green; }
input:disabled { opacity: 0.5; cursor: not-allowed; }
```

## 2.4 Logical / Functional Pseudo-classes

| Pseudo-class            | Description                                                                                    |
|-------------------------|------------------------------------------------------------------------------------------------|
| :not(selector)          | Selects elements that do NOT match the given selector                                          |
| :is(sel1, sel2, ...)    | Matches any element that matches ANY selector in the list (forgiving, low specificity impact)  |
| :where(sel1, sel2, ...) | Same as :is() but always has ZERO specificity                                                  |
| :has(selector)          | Selects an element IF it contains a descendant matching the given selector ('parent selector') |

```
li:not(.active) { opacity: 0.6; }

:is(h1, h2, h3) { font-family: 'Georgia', serif; }

/* Style a <figure> only if it has a <figcaption> inside it */
figure:has(figcaption) { border: 1px solid #ccc; }
```

*:has() is often called the 'parent selector' because it lets you style an ancestor based on what it contains — something CSS could not do before.*

## 2.5 Other Common Pseudo-classes

| Pseudo-class    | Description                                                         |
|-----------------|---------------------------------------------------------------------|
| :target         | Element whose id matches the URL fragment (e.g. page.html#section1) |
| :lang(code)     | Element whose content language matches the given language code      |
| :fullscreen     | Element currently displayed in fullscreen mode                      |
| :dir(ltr / rtl) | Element with a given text direction                                 |
| :scope          | Reference point for scoped selectors (e.g. in querySelector)        |

```
#section1:target {  
background: yellow;  
}
```

```
/* Highlights section when URL is page.html#section1 */
```

## Part 3 — Specificity Quick Notes

Combinators (space, >, +, ~) **do not add specificity** by themselves — they only describe relationships. Specificity is added by the selectors on either side of them (type, class, ID selectors).

Most pseudo-classes count the same as a **class selector** (specificity value 0,1,0), for example :hover, :nth-child(), :not(). Exceptions:

- :where() — always contributes **zero** specificity.
- :is() and :not() — take the specificity of their **most specific argument** (not a flat class value).

```
a.button:hover /* specificity: 0,2,1 */  
ul > li:nth-child(2) /* specificity: 0,1,2 */  
p:where(.intro) /* specificity: 0,0,1 (where() adds nothing) */
```

### Summary Cheat-Sheet

| Category           | Key symbols / keywords                                                        |
|--------------------|-------------------------------------------------------------------------------|
| Combinators        | (space) descendant, > child, + adjacent sibling, ~ general sibling,    column |
| Link/action states | :link :visited :hover :active :focus :focus-visible :focus-within             |
| Structural         | :first-child :last-child :nth-child() :nth-of-type() :only-child :root :empty |
| Form states        | :checked :disabled :enabled :required :valid :invalid :read-only              |
| Logical            | :not() :is() :where() :has()                                                  |
| Other              | :target :lang() :fullscreen :dir()                                            |

---

End of notes — CSS Combinators & Pseudo-classes